GitLab Arbitrary File Read: CVE 2016-9086

Giulio Presaghi

June 9, 2026

1 Introduction

1.1 What's GitLab

GitLab, a Software Forge developed by Dmytro Zaporozhets [Wik26], represents one of the most adopted DevOps platforms within companies and individuals.

Mainly written in Ruby and Go, GitLab stands out with some features that differentiate it from its well known competitors like GitHub and Bitbucket. The principal difference stands in the intended scope of usage. Indeed, while GitHub is a community-based platform and BitBucket is a cornerstone for enterprises, GitLab's core philosophy is to provide every single tool needed for the DevOps lifecycle inside one single software package. It includes native code hosting, CI/CD pipelines, container registries, security scanning (SAST/DAST), package management, and issue tracking. The main benefit, is that you don't need to connect third-party tools; everything works out of the box with unified permissions and user interfaces.

1.2 Architecture for file handling and data migration

For the interest of this report, it is essential to look at how GitLab structures its backend services, how it handles privileged system access, and how data moves through its import/export subsystems.

1.2.1 The Application Layer

At the core of GitLab's web interface and business logic is a Ruby on Rails framework. This layer is responsible for handling all HTTP requests, managing user sessions, enforcing access controls, and orchestrating background jobs. When an administrative or user-facing feature requires file manipulation, such as uploading an avatar, attaching a file to an issue, or migrating data, the Rails backend is the component that receives the data payload and decides how to process it on the local filesystem.

1.3 The project Import/Export subsystem

Introduced with GitLab 8.9 to allow seamless migration between different GitLab instances, the Import/-Export feature relies heavily on file compression.

When a user exports a project, GitLab bundles the project's metadata (issues, merge requests, milestones, and settings) into a series of JSON files, combines them with the raw Git repositories, and packages everything into a compressed tarball archive (`.tar.gz`).

When a user imports a project, they upload this `.tar.gz` file. The Ruby backend accepts the archive, writes it to a temporary directory on the host server, and initiates an extraction process using native Ruby compression libraries to reconstruct the project structure.

1.3.1 High-Privilege service accounts & Shared filesystem access

GitLab is designed to be hosted on Linux servers (or within containers) and executes its processes under a specific, dedicated system user account (typically named git in order to ensure functional integration with the `Git` protocol). Because this service account must manage raw Git repositories, write database configurations, and read application secrets (such as secrets`.yml`, which contains encryption keys and database credentials), the git user possesses broad read and write permissions across critical application directories.

Consequently, any operation executed by the GitLab backend, including unpacking a user-uploaded archive, runs with these exact system privileges. If the application is tricked into reading or writing files outside of its intended temporary directory, the underlying operating system will allow it, provided the git user has access to those files.

1.3.2 Enter CVE 2016-9086

The intersection of these three architectural components created a critical security boundary hazard. The Ruby on Rails backend was given an untrusted, user-supplied archive and told to unpack it using the high-privilege git service account, without checking whether the contents of that archive attempted to break out of the designated temporary import folder.

2 Path Traversal weakness

At its core, CVE-2016-9086 is a Path Traversal (or Directory Traversal) vulnerability kind, formally classified under CWE-22 (Improper Limitation of a Pathname to a Restricted Directory).

The fundamental security failure of a Path Traversal vulnerability is the application's inability to correctly filtering the user input, leading to the possibility of exiting the webroot of the server to access hidden files. In a traditional path traversal attack, an adversary inputs explicit directory-climbing sequences into a file path to escape the webroot.

"By using special elements such as ".." and "/" separators, attackers can escape outside of the restricted location to access files or directories that are elsewhere on the system. One of the most common special elements is the "../" sequence, which in most modern operating systems is interpreted as the parent directory of the current location. This is referred to as relative path traversal. Path traversal also covers the use of absolute pathnames such as "/usr/local/bin" to access unexpected files. This is referred to as absolute path traversal." [MIT26]

2.1 Impact of an attacker's exploit

The consequences of an exploit of Path Traversal span all the three application security areas (the CIA triad). In particular: Integrity is undermined with the attacker being able to overwrite or create critical files, such as programs, libraries, or important data; the Confidentiality is compromised for the attacker may be able to read the contents of unexpected files and expose sensitive data; the Availability is affected for the attacker may be able to overwrite, delete, or corrupt unexpected critical files.

2.2 Dot-dot-slash variant

This is the classic, foundational form of the attack. It relies on how operating systems interpret relative file paths. In both Linux and Windows, two dots (`..`) signify the **parent directory** (one level up).

If an application expects a filename like `report.pdf` and appends it to a hardcoded path like `/var/www/html/`, an attacker will input `../../../../etc/passwd`.

The filesystem resolves `/var/www/html/../../../../etc/passwd` by climbing out of uploads, out of `www`, out of `var`, reaching the root directory (`/`), and entering the `/etc/` folder to read the target file.

2.3 Absolute path traversal variant

Many developers attempt to prevent path traversal by checking for or stripping out `../` sequences. However, they often forget to block absolute paths.

If the application takes a file parameter and passes it directly to a file-reading function, the attacker bypasses the relative path sandbox entirely by providing a direct, absolute path.

Instead of trying to climb up from `/var/www/html/`, the attacker inputs `/etc/passwd` or `C:/Windows/win.ini`. If the backend logic simply hooks the input to the end of a query or overrides the base directory behavior, the operating system jumps straight to the root-level file.

2.4 Archive Decompression Variants (Zip Slip and Tar exploit)

Instead of manipulating a live URL parameter or text form input, this variant hides the path traversal attack inside a compressed file archive (like `.zip`, `.tar.gz`, or `.jar`).

2.4.1 Filename Traversal inside Archives

When creating an archive, file headers store the paths of the files inside them. An attacker manually modifies the archive headers so a file's name is written as `../../../../var/www/shell.php`.

When a vulnerable application unpacks this archive on the server, the decompression utility builds the file path dynamically. It reads the `../` sequences in the filename header, climbs out of the temporary extraction directory and overwrites or drops a malicious file directly into a live web directory.

2.4.2 Symbolic Links (Symlinks) exploit

In Unix-like environments, a symbolic link is a special file that points to another file or directory. An attacker creates a symlink locally that points to a sensitive host file (e.g., link — `/etc/passwd`) and packages it into an archive.

When the server extracts the archive, it blindly recreates the symbolic link on its filesystem. Later, when the application interacts with or reads from that newly extracted folder, it follows the shortcut link, inadvertently traversing out of the sandbox to read or expose the sensitive target file.

2.5 Input Filter Evasion Variant

When applications implement weak, "home-grown" blacklists to block traversal characters, attackers use formatting and encoding tricks to sneak the characters past the security filters.

2.5.1 Double Encoding

Security filters check for standard characters, but the backend web server might decode user input multiple times before passing it to the filesystem.

An attacker replaces the slash (/) or dot (.) with URL encoding. For example, / encodes to `%2f`. If they double-encode it, it becomes `%252f`. The security filter sees `%252f`, thinks it is harmless text, and lets it pass. The backend server then decodes it back to /, executing the path traversal.

2.5.2 Nested Stripping Bypass

If a developer writes a simple sanitization routine that searches for `../` and deletes it once, attackers can nest the sequences inside each other.

The attacker sends the string `....//`. When the application strips out the middle `../`, the remaining characters collapse together: the first two dots (..) join the trailing slash (/), reconstructing a perfect `../` sequence that is then processed by the filesystem.

2.5.3 Null-byte injection

Older language runtimes (like PHP before version 5.3.4 or older C-based frameworks) handle strings using "Null Bytes" (`%00`) to signify the absolute end of a string. Developers would often force a file extension to the end of an input, making a path look like `/html/userinput.pdf`.

An attacker would input `../../etc/passwd%00`. The application would append `.pdf` to it, making it `../../etc/passwd%00.pdf`. However, when passed to the underlying operating system file api, the system stops reading at the Null Byte, completely dropping the `.pdf` extension and opening the raw `/etc/passwd` file.

In the case of this specific CVE, the attacker achieves the exact same outcome using the Symlink Exploitation variant.

Instead of typing traversal characters into a text field, the attacker packages a malicious symbolic link inside a compressed .tar.gz project archive. When GitLab's backend unarchives this file, it fails to validate the file types within the tarball headers. As a result, it writes the symbolic link directly onto the host's disk.

When the application later attempts to read from or re-export that directory, the backend follows the symlink. This acts as a file-system shortcut, forcing the application to traverse completely out of its restricted temporary directory—such as `/var/opt/gitlab/.../tmp/` and directly into the host's root filesystem to access highly privileged files like `/etc/passwd` or GitLab's internal `secrets.yml`.

3 CVE 2016-9086

CVE-2016-9086 is a critical path traversal vulnerability that lives directly within GitLab's Project Import/Export feature, specifically located inside the backend Ruby on Rails source code responsible for parsing and decompressing project migration archives. The specific vulnerable component is the file extraction routine (managed by the `Gitlab::ImportExport::Importer` and `Gitlab::ImportExport::FileImporter` classes [Git26b]), which acts upon user-supplied data transmitted via the file multipart-form parameter during a project import request.

The vulnerability is triggered when an authenticated user uploads a maliciously crafted `.tar.gz` compressed archive containing a symbolic link file record. The flaw manifests because this file-extraction logic relies on unarchiving libraries that blindly process every file entry listed in the archive header without enforcing strict schema or type verification. It specifically fails because the decompression routine does not validate whether an incoming record is a symbolic link rather than a standard flat file or directory, and it fails because the underlying operating system user executing the process, the high-privilege git service account, has broad read and write permissions across the entire host filesystem, allowing the application to inadvertently access restricted paths outside its designated sandbox.

To execute a successful attack scenario, an authenticated user first creates a malicious symbolic link in a local directory using a standard Unix terminal command, such as ln -s /var/opt/gitlab/gitlab-rails/etc/secrets.yml VERSION. This command generates a symlink named `VERSION` (a standard file expected in a GitLab export package) that points directly to the absolute path of GitLab's internal application secrets file on the target server. The attacker then packages this directory using the command `tar -cvzf exploit.tar.gz`, which preserves the symbolic link's pointer reference inside the archive's header data. Next, the attacker logs into the target GitLab instance, navigates to the "**New Project**" dashboard, selects the "**Import project**" from "**GitLab export**" option, and uploads `exploit.tar.gz` via the web interface. When the backend Rails application receives the archive, it saves it to a temporary directory and executes the unpack command; because the code lacks a file-type verification loop to inspect tar headers, it writes the symbolic link `VERSION` directly onto the server's disk, pointing out of the temporary sandbox and straight to the real `secrets.yml`. Finally, the attacker triggers an export of this newly created project or attempts to view the project details through the GitLab UI; the backend process reads the file named `VERSION` on its own local disk, follows the shortcut link, extracts the highly sensitive encryption keys from `secrets.yml`, and delivers the contents back to the attacker's web browser.

By exploiting this flaw, an attacker achieves arbitrary file disclosure, allowing them to read any system configuration or application file accessible to the git user, which ultimately enables them to steal session tokens, forge administrative cookies, access backend databases, and achieve full Remote Code Execution (RCE) on the host machine.

4 Impact and Mitigation

4.1 Audience

This vulnerability affects self-hosted GitLab instances (both Community and Enterprise Editions) running versions 8.9.0 through 8.13.2. The exploit can occur under a precise operational condition: the attacker must possess a valid, authenticated user account on the target GitLab instance. While the underlying path traversal flaw was present since version 8.9.0, it remained localized because only instance administrators could access the migration features. The vulnerability shifted to a critical, easily reachable threat vector in version 8.13.0 when GitLab altered its application configurations to expose the project import utility to all standard registered accounts by default.

4.2 Worst-case impact scenario

The worst-case scenario for a victim organization is a complete host system takeover and data breach. Because the backend Ruby application runs under the privileges of the high-level git operating system user, an attacker can read any configuration file, deployment token, or database credential stored on the filesystem. By extracting secrets.yml, the attacker steals the secret session-signing keys. With these keys, they can forge administrative session cookies, log into the instance as a global administrator, access

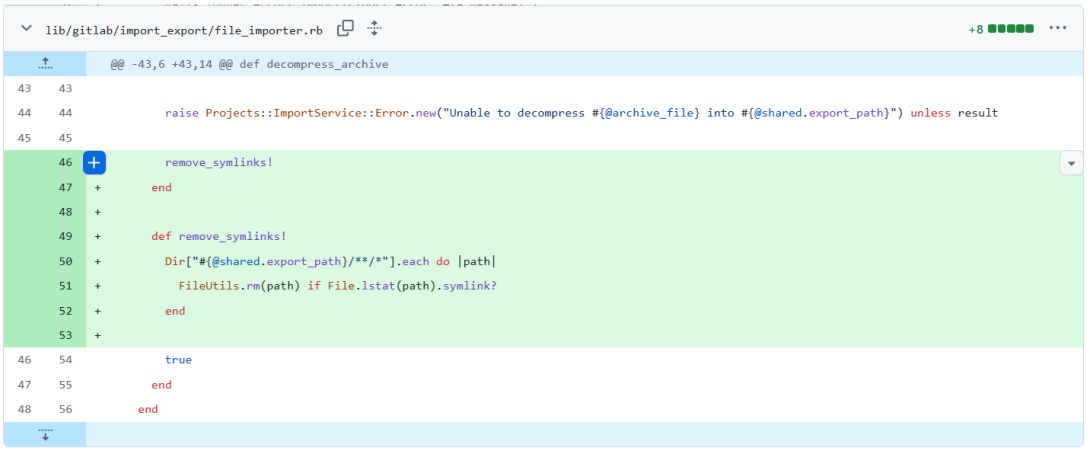

Figure 1: *"Removes any symlinks before importing a project export file. Also added relevant spec."*

every private repository, and pivot to executing arbitrary code (Remote Code Execution) on the server, completely compromising the organization's infrastructure.

4.3 Patch location and technical mechanism

The patch has been applied by Robert Speicher and targeted the entire namespace of GitLab's migration subsystem rather than just a single file [Git26c] [Git26d]. Because the structural vulnerability stemmed from an omission of input validation during archive unpacking, the corrective commits spanned multiple core components, including `lib/gitlab/import_export/file_importer.rb`, third-party migration handlers (such as the Bitbucket and GitHub project importers), and uniform exception-logging modules.

The patch restructured the ingestion pipeline into a strict, two-phase process: an in-memory verification stage and an aggressive post-extraction sanitization stage. To avoid brittle programming practices and the use of hardcoded logics, the codebase abstracts header validation through Ruby's native streaming methods, specifically utilizing `entry.symlink?` and `File.symlink?` checkpoints within a `Gem::Package::TarReader` loop.

The core operational strength of this patch rests on its layered defense-in-depth logic; even if an irregular or malformed archive manages to bypass the initial stream-level headers and momentarily drop a file onto the system, the application immediately invokes dedicated cleanup and removal functions. These remediation routines actively scan the temporary extraction workspace and forcefully execute deletion commands—such as FileUtils.rm1, to purge any symbolic links from the filesystem before any application components or JSON schema parsers can interact with them.

4.4 Why it solves the problem

This code change fundamentally resolves the path traversal flaw by breaking the execution chain before a symbolic link can be created. By utilizing the archiving library's capability to read tar headers directly from the data stream, GitLab checks the typeflag metadata field. Because the application now throws a hard security exception and aborts execution the exact millisecond a symlink record is encountered, the malicious shortcut file is never allowed to exist on the server's local storage. Without the physical symlink file on disk, subsequent application operations cannot be tricked into traversing outside the temporary sandbox directory, permanently neutralizing the file disclosure vector.

5 Exploit Simulation

To reproduce the exploit successfully within the Vulhub Docker environment [Vul26], the attack requires generating a specially crafted archive that tricks GitLab's decompression engine into mapping a system file onto the application web interface. For simplicity, to this report a GitHub repository (inspired by the Vulhub one cited at the beginning) with all the necessary files have been attached [Jul26].

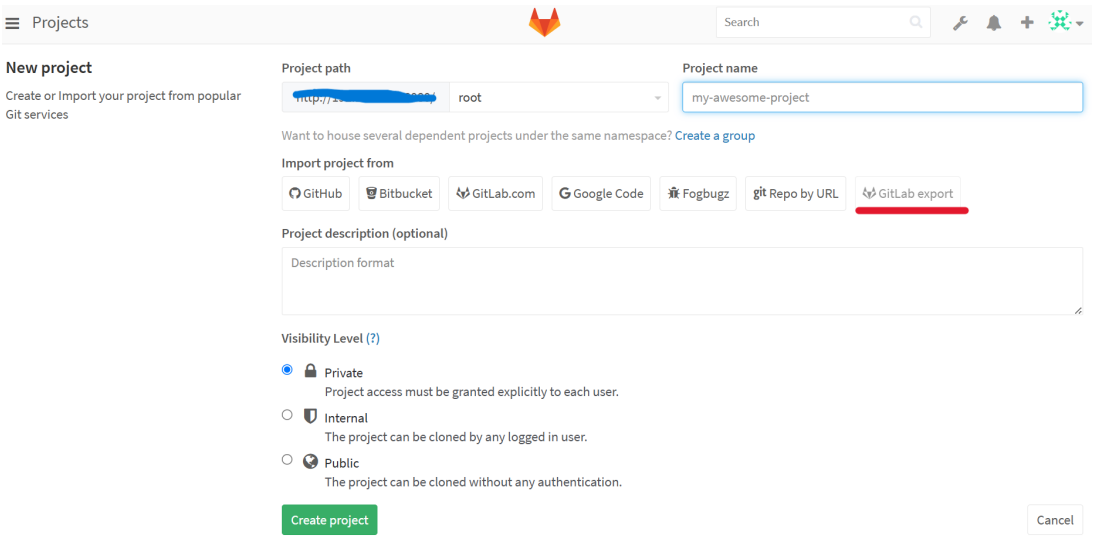

Figure 2: Create a new project, name it and click on GitLab export.

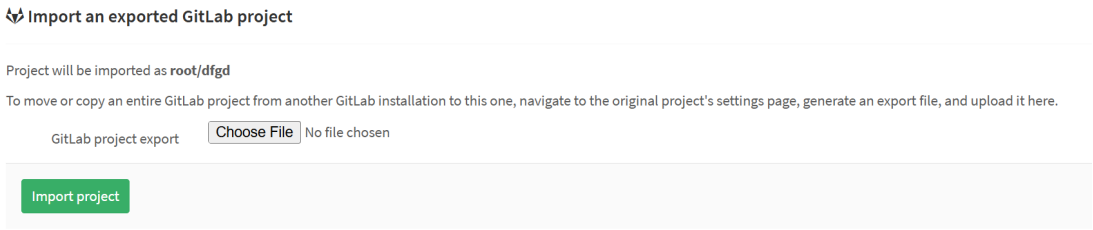

Figure 3: Load the `.tar.gz` file and click on *import project*.

The provided `docker-compose` file, restores GitLab at its 8.13 version when the patch was not yet disclosed.

5.1 Initial Requirements

Make sure to have the latest docker version installed in order to correctly run the docker-compose file.

Make sure to download all the files from the github repo comprising the load-extensions.sh and the test.tar.gz payload.

5.2 Perform the Exploit

Run the docker-compose.yml with this command in the shell or VScode terminal: `docker compose up -d`. After that, authenticate into the running GitLab application container via the browser at http://localhost:8080 (it could take some time due to the heaviness of GitLab), create a new repository instance (2), select the "**GitLab export**" button, and upload the `test.tar.gz` payload inserting a lowercase name in the "**Project name**" field, while leaving the other text fields completely blank to avoid input regex validation naming mismatches (3). Click on "**Import Project**", "**Create New Repository**" and finally, because the Ruby backend lacks file-type restrictions, it unpacks the symlink file directly onto the server's workspace storage; when the application immediately executes its JSON parser to read the project metadata, it follows the newly dropped shortcut link, encounters raw Linux account strings rather than bracketed schema syntax, crashes, and prints the entire extracted file contents straight onto the screen under an unexpected error window (4).

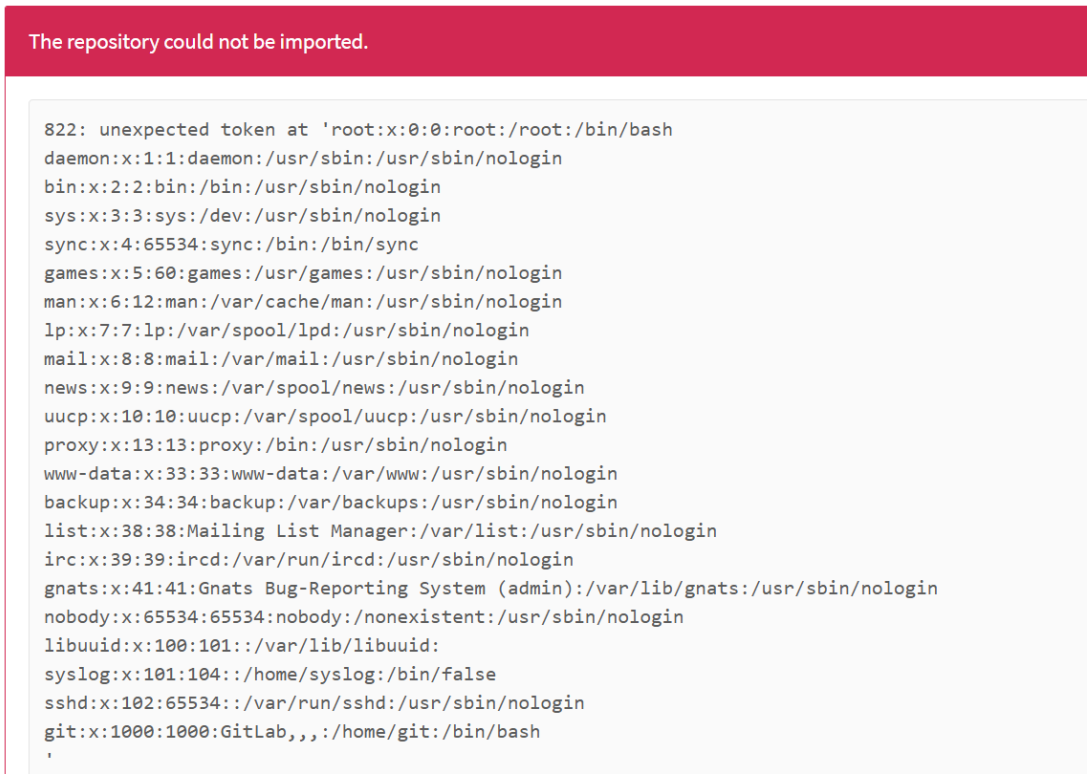

Figure 4: All the extracted file contents are printed as a result of the exploit. All of them are inside the path `/etc/passwd/`

References

[Can26] Canonical Ubuntu Security. CVE-2016-9086 Tracker Details. `https://ubuntu.com/security/CVE-2016-9086`, 2026. [Online; accessed 9-June-2026].

[Git26a] GitLab. GitLab Release Notes. `https://docs.gitlab.com/releases/`, 2026. [Online; accessed 9-June-2026].

[Git26b] GitLab FOSS Contributors. Fix export project file permissions issue (958d9f11). `https://gitlab.com/gitlab-org/gitlab-foss/-/commit/958d9f11ca0e5b7fbcf51f42d2a45a687313bf97`, 2026. [Online; accessed 9-June-2026].

[Git26c] GitLabhq Contributors. Merge branch 'fix/export-att-inclusion' into 'master' (4389f09). `https://github.com/gitlabhq/gitlabhq/commit/4389f09ea17bdf7f191b7e6d0864cf4f3f1e961d`, 2026. [Online; accessed 9-June-2026].

[Git26d] GitLabhq Contributors. Removes any symlinks before importing a project export file (912e1ff). `https://github.com/gitlabhq/gitlabhq/commit/912e1ffcb9e0d16eb7e7b309f3ed1b7145b20642`, 2026. [Online; accessed 9-June-2026].

[Jul26] JulesPress. GitLab-Arbitrary-File-Disclosure: Simulating the exploit and fix of the CVE 2016-9086. `https://github.com/JulesPress/GitLab-Arbitrary-File-Disclosure`, 2026. [Online; accessed 9-June-2026].

[MIT26] MITRE Corporation. CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal'). `https://cwe.mitre.org/data/definitions/22.html`, 2026. [Online; accessed 9-June-2026].

[Nam26] Author Name. Ruby on Rails: Accelerate your agents with convention over configuration. `Insert_URL_Here`, 2026. [Online; accessed 9-June-2026].

[Nat26] National Vulnerability Database (NVD). Official CPE Dictionary Detail: cpe:2.3:a:gitlab:gitlab:8.13.2:*:*:*:community:*:*:*. https://nvd.nist.gov/vuln/detail/CVE-2016-9086, 2026. [Online; accessed 9-June-2026].

[OWA26] OWASP Foundation. Path Traversal. https://owasp.org/www-community/attacks/Path_Traversal, 2026. [Online; accessed 9-June-2026].

[Vul26] Vulhub Contributors. Vulhub vulnerable environment: GitLab arbitrary file read (CVE-2016-9086). https://github.com/vulhub/vulhub/tree/master/gitlab/CVE-2016-9086, 2026. [Online; accessed 9-June-2026].

[Wik26] Wikipedia contributors. Gitlab — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/GitLab, 2026. [Online; accessed 9-June-2026].

[26] /. GitLab (CVE-2016-9086) token . / *Paper*, 2026. [Online; accessed 9-June-2026].